UCS761 - Deep Learning Lab 4

Multiple Linear Regression using a Linear Perceptron

Objective

You will implement **multiple linear regression** using a **single linear neuron (perceptron without activation)** and train it by **minimizing Mean Squared Error (MSE)**.

By the end of this lab, you should understand:

- how numeric prediction differs from classification
- how loss replaces correctness
- how weights change to reduce error
- why this is still the same perceptron at its core

Dataset

Use the following dataset (given officially):

🔗 **Kaggle link**

https://www.kaggle.com/datasets/hussainnasirkhan/multiple-linear-regression-dataset

This dataset contains:

- `YearsExperience`
- `TestScore`
- `Salary`

Environment Choice

You may use **either**:

- **Google Colab** (recommended if Python setup is not ready)

- **Jupyter Notebook** (Anaconda / local setup)

Reason:

- You need basic Python + NumPy + Pandas
- No GPU required
- No deep learning libraries needed

Step 1: Load and Inspect the Dataset

What to do

1. Load the CSV file using Pandas.
2. Print:
    - first 5 rows
    - column names
    - shape of the dataset

Why this step exists

Before modeling, you must know:

- how many inputs you have
- what each column represents
- whether values look numeric and reasonable

Skeleton (fill only what is missing)

```
import pandasas pd

# Load dataset
data = pd.read_csv("YOUR_PATH_HERE")

# Inspect data
print(data.head())
print(data.columns)
```

```
print(data.shape)
```

Think about this (write answers in comments)

- Which columns are inputs?
- Which column is the output?
- How many features does your model need to handle?

Step 2: Separate Inputs and Output

What to do

1. Extract input features into `X`
2. Extract target variable into `y`

Why this step exists

The model must learn a mapping:

$(X1,X2) \rightarrow y$

If inputs and outputs are mixed, learning has no meaning.

Skeleton

```
# Inputs (features)
X = data[["YearsExperience","TestScore"]]

# Output (target)
y = data["Salary"]
```

Think about this

- What is the shape of `X`?
- What is the shape of `y`?

- Why does `X` have 2 columns but `y` only one?

Step 3: Initialize the Model Parameters

What to do

1. Initialize:
    - one weight per input feature
    - one bias term
2. Choose small numeric values

Why this step exists

- Learning starts from **imperfect guesses**
- The model improves them using data
- Initialization should not dominate learning

Skeleton

```
import numpyas np

# Number of features
n_features = X.shape[1]

# Initialize weights and bias
w = np.zeros(n_features)
b =0.0
```

Think about this

- Why do we need one weight per feature?
- Why is bias separate?
- Would initializing with large values be risky?

Step 4: Define the Forward Pass (Prediction)

What to do

Implement the linear model:

y^=w1×1+w2×2+b

Why this step exists

This is the **same perceptron computation** you already know, without a decision function.

Skeleton

```
defpredict(X, w, b):
    """
    X : input features
    w : weights
    b : bias
    returns predicted values
    """
    y_hat = X.dot(w) + b
return y_hat
```

Think about this

- Why is there no activation function?
- What kind of values can `y_hat` take?
- How is this different from logistic regression?

Step 5: Define the Loss Function (MSE)

What to do

Implement **Mean Squared Error**:

L=N1i=1∑N(y^i−yi)2

Why this step exists

The model does not know what “good prediction” means unless you define **how wrong is wrong**.

MSE:

- penalizes large mistakes more
- matches the salary estimation scenario

Skeleton

```
defmean_squared_error(y, y_hat):
    """
    y     : actual values
    y_hat : predicted values
    """
    loss = ((y_hat - y) **2).mean()
return loss
```

Think about this

- Why square the error?
- What happens if one prediction is very wrong?
- Why not just take absolute error?

Step 6: Compute Gradients (Core Learning Step)

What to do

Compute gradients of MSE with respect to:

- weights `w`
- bias `b`

Use the formulas discussed in class.

Why this step exists

This is **where learning happens**.

Without gradients:

- weights never change
- loss never improves

Skeleton

```
defcompute_gradients(X, y, y_hat):
    """
    Compute gradients of MSE loss
    """
    N =len(y)

# Gradient w.r.t weights
    dw = (2 / N) * X.T.dot(y_hat - y)

# Gradient w.r.t bias
    db = (2 / N) * (y_hat - y).sum()

return dw, db
```

Think about this

- Why does `X` appear in `dw` but not in `db`?
- Why does the error term appear everywhere?
- What happens if error is zero?

Step 7: Update Parameters (Gradient Descent)

What to do

Update `w` and `b` using learning rate `lr`.

Why this step exists

- Learning rate controls **how fast** the model learns
- Too large → unstable
- Too small → very slow

Skeleton

```
defupdate_parameters(w, b, dw, db, lr):
    w = w - lr * dw
    b = b - lr * db
return w, b
```

Step 8: Training Loop

What to do

1. Choose:
    - learning rate
    - number of epochs
2. Repeatedly:
    - predict
    - compute loss
    - compute gradients
    - update parameters

Skeleton (structure only)

```
lr =0.0001
epochs =1000
```

```
for epochinrange(epochs):
    y_hat = predict(X, w, b)
    loss = mean_squared_error(y, y_hat)
    dw, db = compute_gradients(X, y, y_hat)
    w, b = update_parameters(w, b, dw, db, lr)

if epoch %100 ==0:
print(f"Epoch {epoch}, Loss:{loss}")
```

Think about this

- Does loss decrease over time?
- What happens if it increases?
- How do learning rate and epochs interact?

Step 9: Final Evaluation

What to do

1. Print:
    - final weights
    - final bias
2. Predict salary for a **new input** (not in dataset).

Example

```
new_candidate = np.array([4.5,68])
predicted_salary = new_candidate.dot(w) + b
print(predicted_salary)
```

Think about this

- Is the prediction reasonable?
- Does it interpolate smoothly?
- Why is this better than threshold rules?

Common Errors (Think, Don’t Panic)

- Loss increases → learning rate too high
- Loss not changing → gradients wrong or lr too small
- Shape errors → mismatch between X, w dimensions
- Predictions exploding → missing normalization (next topic)

> “We didn’t invent a new model.
>
> We removed the decision function and changed the loss.
>
> Everything else stayed the same.”